(1)

Nguyễn Văn Trải 1025DECE085

1,1 Problem 1, Reverse this.

head → 10 → 25 → 90 → 15 → NULL

Solve

Reverse (head):

prev ← NULL

curr ← head

next ← NULL

WHILE (curr ≠ NULL)

(task1)

next ← curr.next

curr.next ← prev

prev ← curr

curr ← next

head ← prev

RETURN head

(task2) 1,25 Pointer trace

iteration	prev	curr	next
initial	NULL	10	—
1	10	25	25
2	25	90	90
3	90	15	15
4	15	NULL	NULL

Linked Stud

Scanned with CamScanner

Initial:

NULL ← 10 | 25 → 40 → 15 → NULL

After iteration 1.

NULL ← 10 | 25 → 40 → 15 → NULL.

After iteration 2.

NULL ← 10 ← 25 | 40 → 15 → NULL

After iteration 3

NULL ← 10 ← 25 ← 40 | 15 → NULL

After iteration 4

NULL ← 10 ← 25 ← 40 ← 15

[head]

Final result : [head] → 40 → 25 → 10 → NULL

Problem 2 : Insert head to a circular LinkedList

Real → 10 → 20 → 30 → (points head)

Solve

Task 1

InsertHead (head, value)

new_node ← create a new node

new_node.value ← value

IF (head = NULL)

new_node.next ← new_node

head ← new_node

RETURN head

tail ← head

Scanned with CamScanner

WHILE (tail.next ≠ head)

tail ← tail.next

new_node.next ← head

tail.next ← new_node

head ← new_node

RETURN head.

1,25 (Task 2)

Initial:

[head]

10 → 20 → 30

right before

Find the tail (iterate ~~until~~ tail == head)

[head] [Tail]

10 → 20 → 30

wire "new_node.next ← head" and ~~"cur.next~~

"tail.next ← new_node"

[head] [tail]

10 → 20 → 30 → new_node

2,5 Problem 3 - Insert before a target (Doubly linked list)

NULL ← A ⇄ B ⇄ C → NULL

(Task 1) 0,75

Insert before (head, target, value):

curr ← head

WHILE (curr ≠ NULL AND curr.value ≠ target)

curr ← curr.next

IF curr NULL:

RETURN head

Scanned with CamScanner

new_node ← creat a new node

new_node.value ← value

IF curr = head:

new_node.prev ← NULL

new_node.next ← curr

curr.prev ← new_node

head ← new_node

ELSE:

new_node.prev ← curr.prev

new_node.next ← curr

curr.prev.next ← new_node

curr.prev ← new_node

Return head

(Task 2) Q75

Let's say (target)

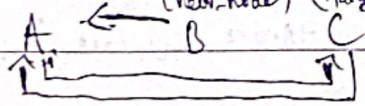
A ⇄ B

and we want:

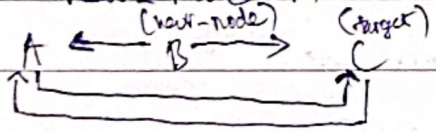
A ⇄ B ⇄ C (target)

(new_node)

First new_node.prev ← curr.prev



Then new_node.next ← curr



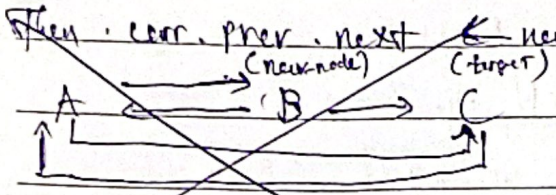
Scanned with CamScanner

(2)

Nguyễn Văn Trải N/25 DECE 085

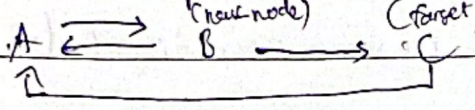
(Task 2 (continue))

Then curr.prev.next ← new_node

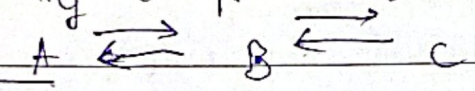


Finally curr.prev

Then curr.prev.next ← new_node



finally curr.prev ← new_node



100 (Task 3) Edge cases.

Case 1: target is head (head)

A B (insert before) B ⇄ C

new_node.prev ← NULL

new_node.next ⇔ head

head.prev ⇔ new_node

head ← new_node

Result: A ⇄ B ⇄ C (A becomes the new head)

Case 2: Target doesn't exist (target = B)

A ⇄ B ⇄ C

We have

WHILE (curr ≠ NULL AND curr.value ≠ target):

curr ← curr.next

ĐOÀN KẾT - TRÁCH NHIỆM - CHẤT LƯỢNG - TIÊN PHONG

Scanned with CamScanner

This will result curr = NULL

IF curr = NULL:

RETURN head

So the list remain unchanged.

2.5 Problem 4 Insert Sorted

2 → 5 → 9

Insert 6 and receive ascending order

1.25 (Task 1)

SortedInsert (head, value)

new_node ← create new node;

new_node.value ← value

IF (head = NULL OR value < head.value)

new_node.next ← head

head ← new_node

RETURN head.

prev ← NULL

curr ← head

WHILE (curr ≠ NULL AND curr.value < value)

prev ← curr

curr ← curr.next

new_node.next ← curr

prev.next ← new_node.

RETURN head

2.25 (Task 2)

Twitter

Scanned with CamScanner

(head)

2 → 5 → 8 → NULL

Set:

prev = NULL

curr = 2.

Check: (Iteration 1)

curr.value 2 < 6 : TRUE.

Move

prev = 2

next = 5

State : 2 → 5 → 8 → NULL

(prev) (curr)

Check : (Iteration 2)

curr.value 5 < 6 : TRUE

Move

prev = 5

next = 8

State : 2 → 5 → 8 → NULL

(prev) (curr)

Check : (Iteration 3)

curr.value 8 < 6 : FALSE

Stop.

Insert new_node right before curr

new_node.next ← curr

prev.next ← new_node

ĐOÀN KẾT - TRÁCH NHIỆM - CHẤT LƯỢNG - TIÊN PHONG

Scanned with CamScanner

Check

2 → 5 → 6 → 8 → NULL

(head) (prev) (new_node) (curr)

Complexity

Problem	time	space
1	O(n)	O(1)
2	O(n)	O(1)
3	O(n)	O(1)
4	O(n)	O(1)

Problem 1 : 2,75

Problem 2 : 2,5

Problem 3 : 2,5

Problem 4 : 2,5

9,75

Scanned with CamScanner